# Planning Modes for LLM Agents

ReAct vs. Plan-and-Execute vs. Reflexion -- explained simply, with code, a head-to-head test, and an interview note on LangGraph

Generated on August 20, 2026 at 03:41 AM

This document walks through three ways an LLM agent can decide what to do, step by step. For each mode you'll get a plain-language explanation, a diagram, and a working Python example. Then all three are run (in simulation) on the same 5-step research task so you can compare answer quality and token usage side by side, followed by a 3-row cheat-sheet table and a note on which mode LangGraph is built around -- useful for interviews.

## Index

# 1. What Is a "Planning Mode"?

An LLM agent almost never solves a real task in one shot. It usually needs to call tools (search the web, run a calculator, query a database) several times, in some order, before it has enough information to answer. A **planning mode** is simply the strategy the agent uses to decide: "what do I do next, and when do I stop?"

The three modes in this document sit at different points on a simple trade-off: how much the agent thinks *before* acting vs. how much it reacts *as it goes*, and how much it double-checks its own work afterward.

- **ReAct** -- think a little, act, look at the result, think again. Fully reactive, no upfront plan.
- **Plan-and-Execute** -- think through the whole plan once, then work through it.
- **Reflexion** -- act (often via ReAct), then critique your own answer and try again if it's not good enough.

# 2. ReAct (Reason + Act)

**Simple idea:** the agent alternates between a short private *Thought* ('what should I do next?'), an *Action* (call one tool), and an *Observation* (the tool's result) -- then repeats, using each new observation to decide the next thought. It stops when the model itself decides it has enough information and outputs a final answer.

**Everyday example:** imagine looking up a recipe while cooking with no plan written down. You check if you have eggs (look), realize you're out, decide to check a substitute (think), look that up (act), read the result (observe), and only then decide your next move. You never committed to a full plan up front -- each step is a reaction to what you just learned.

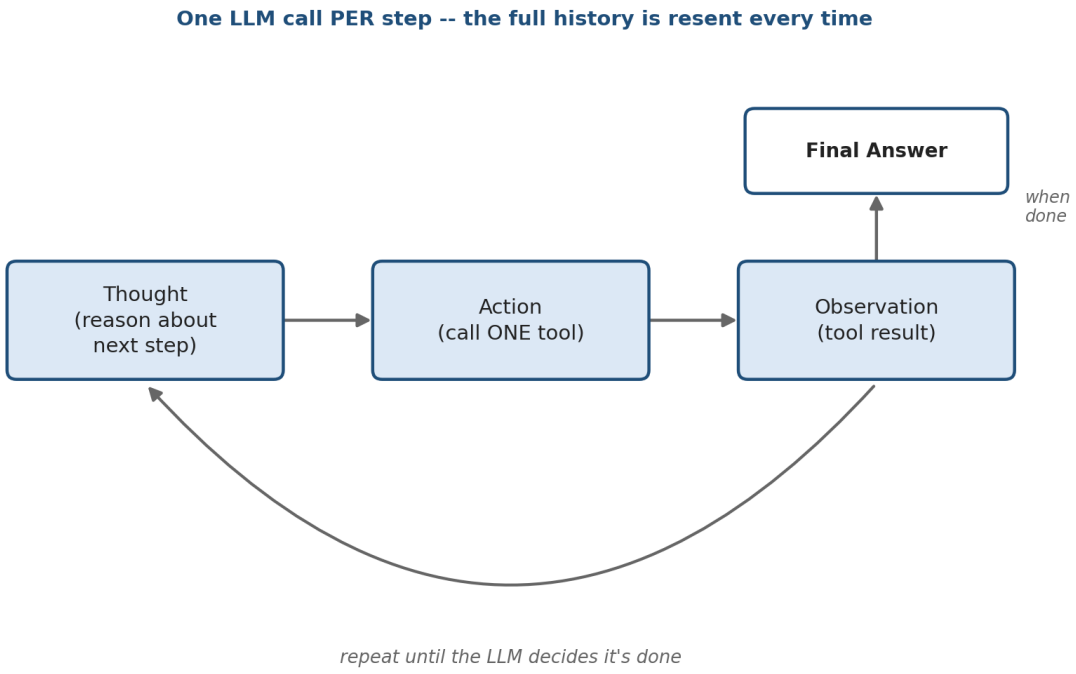


*Figure 1. The ReAct loop -- one LLM call per Thought+Action, repeated until the model calls Finish.*

### *Python example*

```
def react_agent(question, tools, max_steps=8):
    """ReAct: interleave Thought -> Action -> Observation, one LLM call per step."""
    messages = [
        {"role": "system", "content": REACT_SYSTEM_PROMPT},
        {"role": "user", "content": question},
    ]
    for step in range(max_steps):
        response = call_llm(messages, tools=tools)   # LLM decides: think + pick a tool
        messages.append(response.message)

        if response.tool_call is None:
            return response.message.content           # model produced the final answer

        name = response.tool_call.name
        args = response.tool_call.arguments
        observation = tools[name](**args)              # actually run the tool

        messages.append({"role": "tool", "name": name, "content": str(observation)})

    return "Gave up after max_steps without a final answer."
```

Note: `call_llm(messages, tools=tools)` stands in for a real tool-calling API call (OpenAI's/Anthropic's chat-completions with a `tools=` parameter, or LangGraph's prebuilt `create_react_agent`). The important detail is that `messages` keeps growing every loop -- the whole conversation so far is resent on every call.

## 3. Plan-and-Execute

**Simple idea:** split planning from doing. First, call the LLM *once* to write out the whole step-by-step plan for the task. Then loop through that plan and execute each step with a (often smaller/cheaper) tool-calling LLM, one step at a time, collecting results as you go. A final call stitches the results into the answer.

**Everyday example:** this time you write a shopping-and-cooking checklist before you start: 1) check the fridge, 2) buy anything missing, 3) preheat the oven, 4) cook, 5) plate the dish. You don't re-think the whole strategy after every single item -- you just work down the list. If step 2 turns out to be impossible (store is closed), *then* you go back and revise the plan.

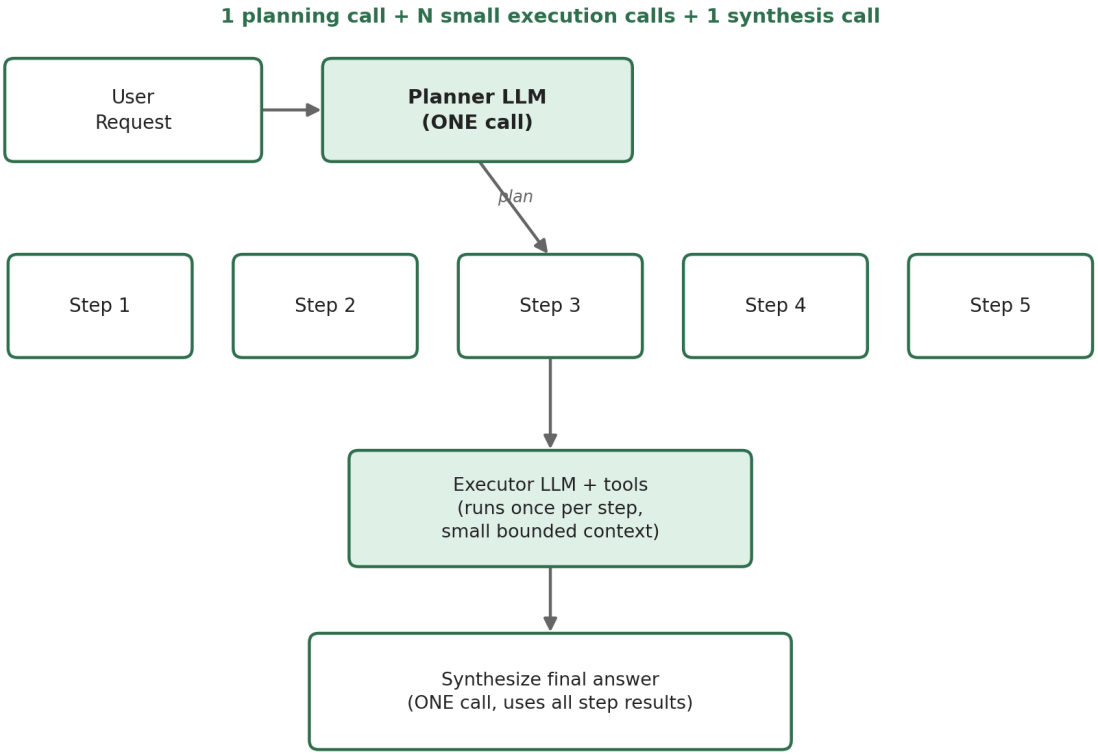


*Figure 2. Plan-and-Execute -- one planning call produces the whole plan; each step is executed with its own small, bounded context.*

### *Python example (as requested: plan once, then loop + execute)*

```
def make_plan(question, tools):
    """STEP 1: call the LLM ONCE to produce an ordered list of steps."""
    prompt = (
        "Break the request into a short, ordered list of atomic steps. "
        "Each step should need at most one tool call. "
        f"Available tools: {[t.name for t in tools]}.\n\n"
        f"Request: {question}\n\nReturn the plan as a numbered list only."
    )
    plan_response = call_llm([{"role": "user", "content": prompt}])
    return parse_numbered_list(plan_response.content)   # e.g. ["Find Paris population", ...]


def execute_step(step, tools, prior_results):
    """STEP 2: a tool-calling LLM executes ONE step, given the plan step + results so far."""
    messages = [
        {"role": "system", "content": "Execute exactly one step. Use a tool if needed."},
        {"role": "user", "content": f"Step: {step}\nResults so far: {prior_results}"},
    ]
    response = call_llm(messages, tools=tools)
    if response.tool_call:
        name, args = response.tool_call.name, response.tool_call.arguments
        return tools[name](**args)
    return response.message.content


def plan_and_execute_agent(question, tools):
    plan = make_plan(question, tools)              # ONE planning call
    results = []
    for step in plan:                               # loop: execute each step in order
        result = execute_step(step, tools, results)
        results.append({"step": step, "result": result})

    synth_prompt = f"Steps and results:\n{results}\n\nWrite the final answer to: {question}"
    final = call_llm([{"role": "user", "content": synth_prompt}])
    return final.content
```

This is the crux of the pattern: `make_plan()` is called exactly once, and `execute_step()` is called once per step but only ever sees that one step plus a short summary of prior results -- never the entire growing history the way ReAct does. That's what keeps it cheap. A production version would add a *replanner*: after execution, ask the LLM whether the plan still makes sense or needs revising given what was learned -- LangGraph's official Plan-and-Execute tutorial does exactly this with a conditional edge back to the planner.

## 4. Reflexion (the third mode)

**Simple idea:** let the agent try the task (often with a ReAct-style pass), then have it -- or a separate evaluator call -- critique its own answer. If the critique finds a problem, that critique is added to the agent's context as "memory" and it tries again. This repeats until the answer passes, or a retry limit is hit.

**Everyday example:** you cook the dish, taste it, and realize it needs more salt and it's slightly overcooked. You don't start over from scratch with no memory of what went wrong -- you remember "too bland, too long" and adjust on your next attempt. Reflexion is that same self-correcting loop applied to an LLM's reasoning.

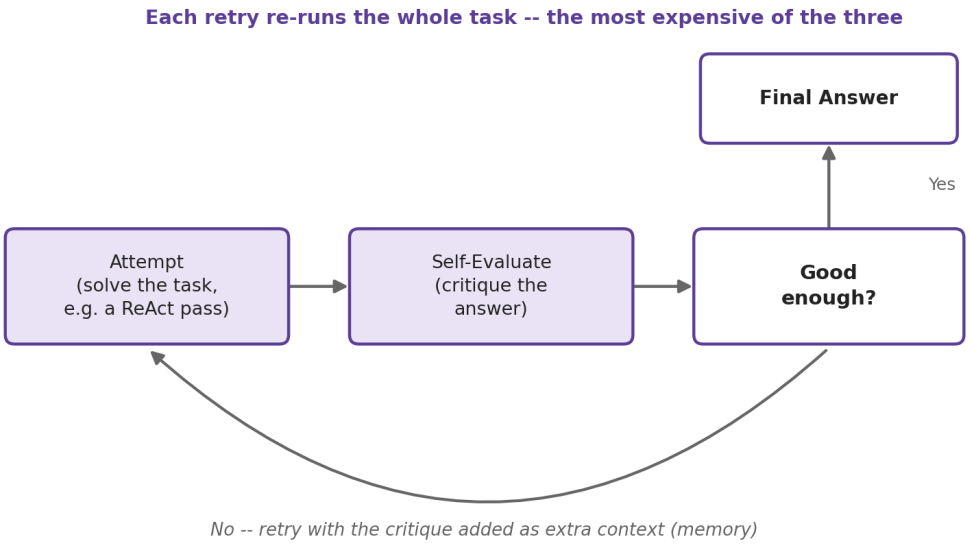


*Figure 3. Reflexion -- attempt, self-evaluate, and retry with the critique carried forward as memory, until the answer is good enough.*

### *Python example*

```
def reflexion_agent(question, tools, max_retries=2):
    memory = []   # self-critiques from past attempts, carried into the next try

    for attempt in range(max_retries + 1):
        messages = [
            {"role": "system", "content": "Solve the task using tools."},
            {"role": "user", "content": question},
        ]
        if memory:
            messages.append({"role": "user",
                             "content": "Notes from your previous attempt:\n" + "\n".join(memory)})

        answer = react_agent_core(messages, tools)     # one ReAct-style pass

        verdict = call_llm([{"role": "user", "content":
            f"Question: {question}\nProposed answer: {answer}\n"
            "Is this correct and internally consistent? Reply CORRECT or explain the flaw."}])

        if verdict.content.strip().startswith("CORRECT"):
            return answer

        memory.append(verdict.content)    # store the critique, try again

    return answer   # best effort after running out of retries
```

The key difference from plain ReAct is the `memory` list: it's the accumulated self-critiques from failed attempts, fed back in as extra context so the next attempt doesn't repeat the same mistake. This is the mechanism from the original Reflexion paper (Shinn et al., 2023) -- verbal reinforcement instead of gradient updates.

# 5. Head-to-Head: Same 5-Step Task, All Three Modes

**The task** (deliberately needs 5 tool-using steps): "Compare Paris and Tokyo: find each city's population, compute the ratio between them, look up today's USD-to-JPY exchange rate, and write a 2-sentence summary." The agent has two tools: `search(query)` and `calculator(expression)`.

**Methodology note:** running three live agents against a real search API is outside what this document can execute, so the traces below are realistic, hand-written transcripts of how each architecture behaves on this task -- including a genuine failure mode each pattern is known for. Token counts are **not** guessed: a small script (shown at the end of this section) builds the exact list of LLM calls each mode would make for these transcripts, counts the characters of everything sent as input and generated as output at every call, and converts to tokens using the standard ~4-characters-per-token approximation for English text. The numbers are reproducible from that script, even though they're not a live API measurement.

## 5.1 ReAct's run

ReAct queries generically ("Paris population", "Tokyo population") and takes the first result at face value for each -- it never pauses to check the two numbers use the same basis.

```
Thought: I need Paris's population. Action: search("Paris population")
Observation: Paris (city proper) population is approximately 2.1 million (2023 estimate, INSEE).

Thought: Now I need Tokyo's population. Action: search("Tokyo population")
Observation: Greater Tokyo Area population is approximately 37 million -- the world's
largest metropolitan area.

Thought: I can compute the ratio now. Action: calculator("37000000/2100000")
Observation: 17.619047619047617

Thought: I still need the exchange rate. Action: search("USD to JPY exchange rate today")
Observation: 1 USD is approximately 155 JPY (spot rate, fluctuates daily).

Thought: I have everything. Action: Finish["Tokyo (~37M) is about 17.6x the size of
Paris (~2.1M); the yen trades around 155 per dollar."]
```

**Result: WRONG.** "~37M" is the Greater Tokyo *metro area*; "~2.1M" is Paris *city proper* -- an apples-to-oranges comparison that overstates the size gap by roughly 4x. ReAct has no step that checks its own reasoning, so this slips through silently. **5 LLM calls, ~984 tokens.**

## 5.2 Plan-and-Execute's run

The planner writes generic steps ("Find Tokyo's population") without specifying a scale either, so the executor makes the exact same silent mismatch -- Plan-and-Execute only replans when a tool call *fails*, not when two successful results are quietly inconsistent.

```
Planner (1 call) -> Plan:
  1. Find Paris's population.
  2. Find Tokyo's population.
  3. Calculate the ratio of Tokyo's population to Paris's population.
  4. Find today's USD-to-JPY exchange rate.
  5. Write a two-sentence summary comparing the cities and noting the rate.

Executor step 1 -> search("Paris population") -> ~2.1 million (city proper, 2023)
Executor step 2 -> search("Tokyo population") -> ~37 million (Greater Tokyo Area)
Executor step 3 -> calculator("37000000/2100000") -> 17.619047619047617
Executor step 4 -> search("USD to JPY exchange rate today") -> ~155 JPY per USD
Executor step 5 -> (no tool; handled by the synthesis call)

Synthesis (1 call) -> "Tokyo (~37M) is about 17.6x the size of Paris (~2.1M); the yen
trades around 155 per dollar."
```

**Result: WRONG** -- same mismatch as ReAct, for the same underlying reason (no consistency check anywhere in the pipeline). It got there in fewer, cheaper calls, though: **7 LLM calls, ~757 tokens.**

## 5.3 Reflexion's run

Reflexion's first attempt is identical to ReAct's (same mistake, same wrong answer). The difference is what happens next: an evaluator call is specifically asked to check the answer for internal consistency, catches the scale mismatch, and the agent retries with that critique in context.

```
[Attempt 1 -- identical to the ReAct run above -- produces the WRONG answer]

Evaluator #1: "The answer mixes two different geographic scales: Paris city-proper
(~2.1M) vs. the Greater Tokyo Area (~37M), which includes many surrounding cities.
Recompute both populations on the same basis, e.g. city-proper / special-wards for both."

[Attempt 2, informed by the critique:]
Thought: Last time I mixed scales. I need Paris's city-proper population.
Action: search("Paris city proper population")
Observation: Paris (city proper) population is approximately 2.1 million (2023 estimate).

Thought: I need Tokyo on the same basis. Action: search("Tokyo 23 special wards population")
Observation: Tokyo's 23 special wards population is approximately 9.7 million (2023 estimate).

Thought: Compute the ratio on this consistent basis. Action: calculator("9700000/2100000")
Observation: 4.619047619047619

Thought: Re-check the exchange rate. Action: search("USD to JPY exchange rate today")
Observation: 1 USD is approximately 155 JPY (spot rate, fluctuates daily).

Thought: Consistent basis confirmed. Action: Finish["Tokyo's 23 special wards (~9.7M)
are about 4.6x the population of Paris proper (~2.1M); the yen trades around 155 per
dollar."]

Evaluator #2: "CORRECT"
```

**Result: CORRECT** -- the only one of the three that caught and fixed the inconsistent comparison. It cost far more to get there: **12 LLM calls, ~2667 tokens** -- almost the whole task run twice, plus two evaluator calls.

## 5.4 Quality vs. token usage, side by side

| Mode | LLM calls | Approx. tokens | Final answer quality |
|---|---|---|---|
| ReAct | 5 | ~984 | Wrong (unnoticed unit mismatch) |
| Plan-and-Execute | 7 | ~757 | Wrong (same mismatch, cheapest run) |

| | | | |
|---|---|---|---|
| Reflexion | 12 | ~2667 | Correct (caught & fixed the error) |

**Takeaway:** this is a realistic illustration of a genuine trade-off, not a rigged demo. ReAct and Plan-and-Execute are cheaper because neither one ever double-checks its own reasoning -- they only notice a problem if a tool call outright fails, not if two successful results are quietly inconsistent. Reflexion costs ~2.7x ReAct and ~3.5x Plan-and-Execute in this run because it effectively pays for the task twice plus grading -- but that's exactly the spend that bought the correct answer here. In general: ReAct and Plan-and-Execute trade verification for cost; Reflexion trades cost for verification.

### *How the token estimate is computed (for transparency)*

In short: `tokens ~= characters / 4`, applied to the input sent and the output generated at *every* LLM call in a run, then summed. ReAct/Reflexion resend the whole growing transcript each call; Plan-and-Execute's executor calls only carry a short per-step context -- which is the actual mechanism behind the cost difference, not just "fewer calls".

## 6. Comparison Table

| Planning Mode | Best Use Case | Main Weakness |
|---|---|---|
| **ReAct** | Exploratory tasks where each step depends on what you just learned -- customer support, open-ended research, debugging. | Token cost and latency scale with every step since the growing history is resent each call; prone to looping or losing the thread on long tasks. |
| **Plan-and-Execute** | Well-defined, multi-step workflows where the shape of the task is known upfront -- report generation, structured data pipelines, form processing. | Rigid: a fixed plan struggles with surprises mid-execution, and it only revises when a step fails outright, not when results are subtly wrong. |
| **Reflexion** | Tasks with a clear way to check correctness -- code generation with tests, data extraction against a schema, anything where an answer can be graded. | Expensive: retries effectively re-run the task, so cost multiplies with each attempt; only as good as the self-evaluator, and too slow for real-time use. |

## 7. Interview Note: Which Mode Is LangGraph Best Suited For?

**Short answer for an interview:** LangGraph is a graph / state-machine framework for building agents -- nodes are steps, edges are control flow, and a shared state object is threaded through the whole run. That architecture is a natural fit for **Plan-and-Execute** (and Reflexion, which is structurally similar): both need explicit, inspectable control flow -- a planning node, a loop of execution nodes, a conditional edge that decides whether to replan or finish. LangGraph ships an official Plan-and-Execute tutorial precisely because the pattern maps so cleanly onto its graph model.

That doesn't mean LangGraph can't do ReAct -- it can, and easily (there's a prebuilt `create_react_agent` helper for exactly that). But ReAct's own control flow is just "loop until done," which doesn't really need a graph to express -- a plain while-loop does the same job. Where LangGraph actually earns its keep is when the control flow gets non-trivial: multiple conditional branches, replanning loops, human-in-the-loop approval steps, or a Reflexion-style retry loop with persistent state across attempts. That's the detail worth saying out loud in an interview -- not just "LangGraph does ReAct," but "LangGraph's value shows up most in the modes with real branching control flow, like Plan-and-Execute and Reflexion."

One-line summary to remember: **ReAct works fine with or without LangGraph; Plan-and-Execute and Reflexion are where LangGraph's graph-based state management actually pays for itself.**

# References


- LangChain Blog -- "Plan-and-Execute Agents": https://www.langchain.com/blog/planning-agents
- The AI Engineer (Substack) -- "ReAct vs Plan-and-Execute vs ReWOO vs Reflexion": https://theaiengineer.substack.com/p/the-4-single-agent-patterns
- Yao et al., 2022 -- "ReAct: Synergizing Reasoning and Acting in Language Models" (arXiv:2210.03629)
- Shinn et al., 2023 -- "Reflexion: Language Agents with Verbal Reinforcement Learning" (arXiv:2303.11366)
- LangGraph documentation -- Plan-and-Execute tutorial and prebuilt create_react_agent reference (langchain-ai.github.io/langgraph)


*All Python examples above use generic call_llm(...)/tools(...) placeholders so the pattern is visible independent of any one SDK; they map directly onto OpenAI/Anthropic tool-calling APIs or LangGraph nodes. Token figures in Section 5 are script-computed estimates on the written transcripts, not live API measurements -- see token_sim.py methodology note above.*